PROGINEER · BACKEND PRACTICAL ASSIGNMENT · TASK 2 REPORT

Showroom API with Git Workflow & CI/CD

Izzaldeen · November 2026 · ASP.NET Core (.NET 10) · EF Core 10 · SQL Server · JWT · xUnit + Moq · GitHub Actions · Docker
Repository: github.com/aldinizz2004-png/ShowroomManagement-Testing

190	99.4 %	6 PRs	2 jobs	Staging
tests passing (111 unit + 79 integration)	line coverage · 92 % branch	feature branches, review, 1 merge conflict	Build & Test on every push/PR, then deploy	image on ghcr.io, smoke test passed

1 What Task 2 adds to the Task 1 API

Task 1 delivered the vehicle inventory with unit and integration tests. Task 2 builds on the same solution and the same layers (`Controller → Service → IRepository → Repository → DbContext`):

Customer management
Add, update, view and list (with `?search=`). Fields: full name, unique email, phone, address. A profile can be linked to one login account: staff add walk-in customers, and `/api/auth/register` creates a Customer account with its profile (or links an existing walk-in profile with the same email).

Reservations
Customer, vehicle, reservation date (set by the server), expected purchase date, status (`Active` , `Cancelled` , `Completed`). Create, cancel, view and list, plus "complete sale" for staff.

Role-based and resource-based access

Operation	Admin	SalesRep	Customer	No token
Vehicles: list / view	all	all	available only	401
Vehicles: add / update / status	✓	✓	403	401
Vehicles: delete	✓	403	403	401
Customers: add / update / view / list	✓	✓	403	401
Reservations: create	any customer	any customer	for themselves	401
Reservations: view / list / cancel	all	all	own only (403 otherwise)	401
Reservations: complete sale	✓	✓	403	401

Role-based (policies)
Named policies `ManageVehicles` , `DeleteVehicles` , `ManageCustomers` , `ManageReservations` on controllers, plus a fallback policy that requires a signed-in user on every endpoint.

Resource-based (ownership)
"Is this reservation yours?" can only be answered after loading it, so `ReservationService` compares the reservation's customer with the user id from the JWT (`CurrentUser`). A customer's list is always filtered to their own profile.

Reservation rules

Rule	Behaviour
Vehicle must be `Available` (not Reserved / Sold / missing)	400; reserving sets the vehicle to `Reserved`
Expected purchase date	from today up to 90 days, otherwise 400 (boundaries tested with a fixed `TimeProvider`)
Cancel (only `Active`)	status `Cancelled` , vehicle back to `Available`
Complete sale (staff, only `Active`)	status `Completed` , vehicle becomes `Sold`
One active reservation per vehicle	also enforced in the database by the filtered unique index `IX_Reservations_OneActivePerVehicle`
Vehicle with reservations	cannot be deleted (400); it is marked as Sold instead

Each change to a reservation and its vehicle is saved in one `SaveChanges` call, so both are updated together. Services return `ServiceResult<T>` (Ok / Validation / NotFound / Forbidden) and `ApiControllerBase` maps it to 200 / 400 / 404 / 403.

2 Extended tests

Unit tests (xUnit + Moq)	Tests
`ReservationServiceTests` new	27
`VehicleServiceTests` (+ delete with reservations)	25
`CustomerServiceTests` new	15
`AuthServiceTests` (+ profile linking)	7
`AuthorizationPolicyTests` (+ 2 policies)	5
`TokenServiceTests`	3
Methods above; `[Theory]` cases expand them to **111** test runs.	

Integration tests (WebApplicationFactory)	Tests
`ReservationApiTests` new	14
`CustomerApiTests` new	10
`CustomerVehicleVisibilityTests` new	4
`HealthCheckTests` new	1
`VehicleCrudTests`, `VehicleAuthorizationTests`	27
`AuthApiTests`	6
Methods above; with `[Theory]` cases: **79** test runs.	

Unit level: business rules in isolation

Reserving an available vehicle, reserved / sold / missing vehicles, past dates and dates beyond 90 days (0 and 90-day boundaries), a customer reserving for someone else (Forbidden), staff without a customer id, cancel / complete rules and vehicle status changes, duplicate customer email, invalid email and phone. Repositories are Moq fakes, and `Verify` checks that nothing is saved when a rule fails.

Integration level: the real HTTP pipeline

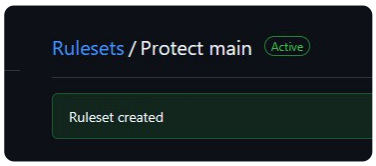
The API runs in memory with SQLite and real JWT logins per role. The full flow is tested end to end: reserve → vehicle Reserved → cancel → Available; complete → Sold. Also 401 without a token, 403 for the wrong role or another customer's reservation, 404, 400 for invalid data, customers seeing only available vehicles, and `/health`.

Coverage (coverlet + ReportGenerator, API project, migrations excluded): **99.4 %** lines (366/368) and **92 %** branches combined. Every service, validator, policy and the token service is at 100 % from unit tests alone.

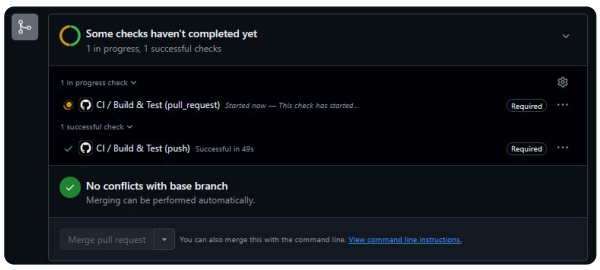
3 Git workflow

Rules: one branch per feature, small commits with clear messages, one pull request per branch, branch deleted after merge. `main` is protected by the ruleset **Protect main**: changes only through a pull request, the **Build & Test** check must pass before merging, and force pushes and deletion are blocked.

PR	Branch	Content	Commits
#1	`feature/ci-pipeline`	CI workflow; review comment fixed (NuGet cache)	2
#2	`feature/customers`	Customer module, registration linking, customer view of vehicles	5
#3	`feature/reservations`	Reservations with ownership checks	5
#4	`feature/staging-deploy`	`/health`, Dockerfile, deploy to staging	3
#5	`docs/readme-title`	README title (conflict setup)	1
#6	`docs/readme-task2`	README + coverage; conflict resolved	2

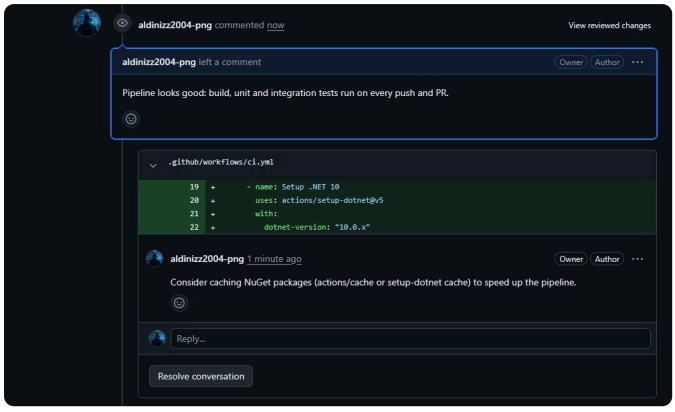


Ruleset "Protect main" is active.

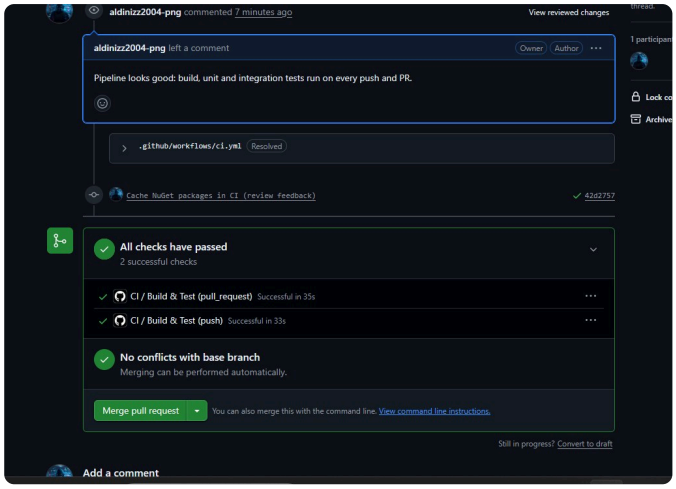


PR #2: the merge button stays disabled while the required check is still running.

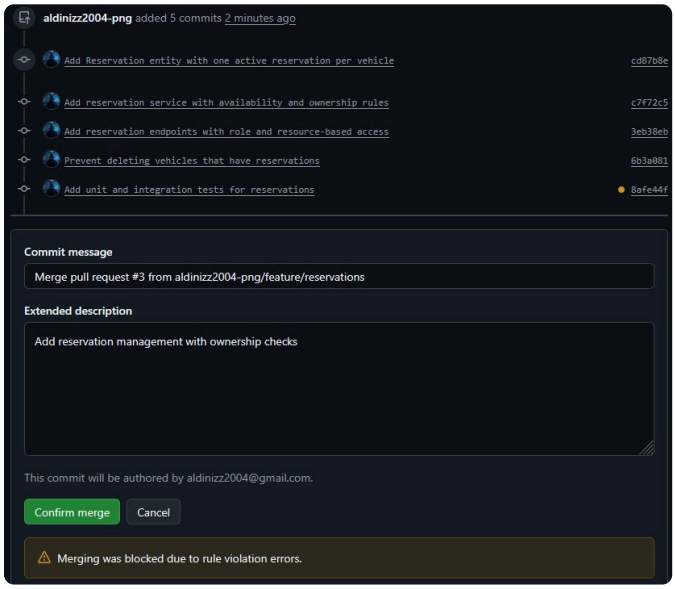
4 Code review and the merge-protection rule



PR #1 review: a line comment on `ci.yml` asks for NuGet caching.



After the fix: commit "Cache NuGet packages in CI (review feedback)" was pushed, CI ran again, and the conversation was resolved.



PR #3: "Merging was blocked due to rule violation errors". The merge was attempted before the required check finished, so the ruleset rejected it. It was merged once the check passed.

Review cycle used

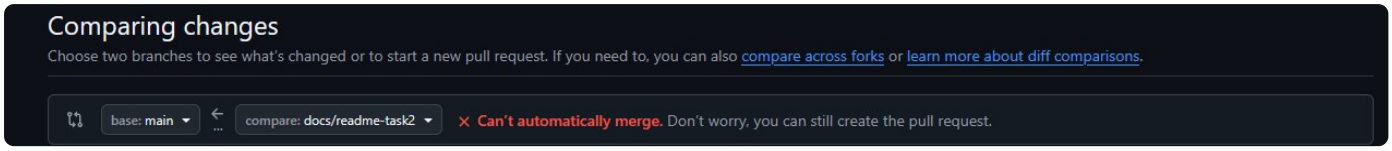
1. Open the PR with a description (what, steps, how to verify).
2. Reviewer leaves a comment on a specific line.
3. Author pushes a fix commit to the same branch.
4. CI re-runs automatically on the new commit.
5. Reply and **Resolve conversation**, then merge.

Why the rule matters

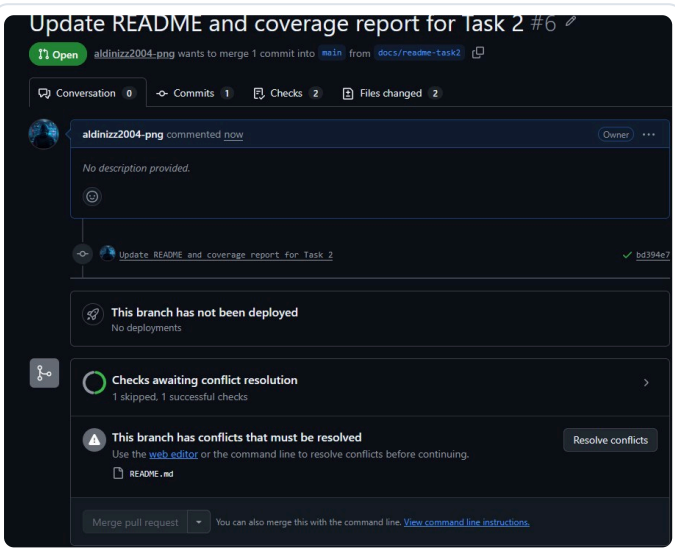
Without it, a broken build could be merged by mistake. With it, `main` only ever receives code that compiled and passed all 190 tests, so it stays stable and deployable.

5 Simulated merge conflict

Two branches changed the **same line** of `README.md` (the title). PR #5 (`docs/readme-title`) was merged first, so PR #6 (`docs/readme-task2`) could no longer be merged automatically.



Opening PR #6: "Can't automatically merge".

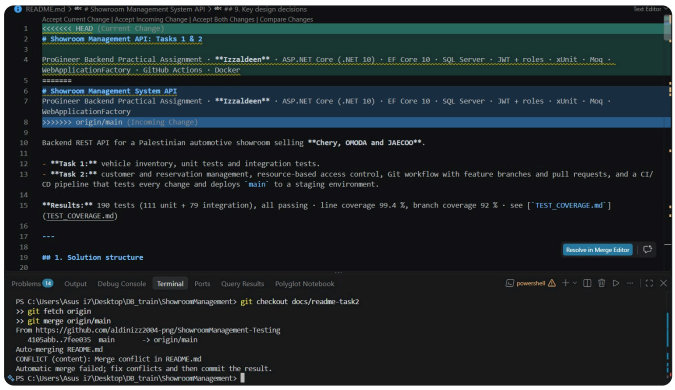


PR #6: "This branch has conflicts that must be resolved" in `README.md` ; checks wait for the resolution.

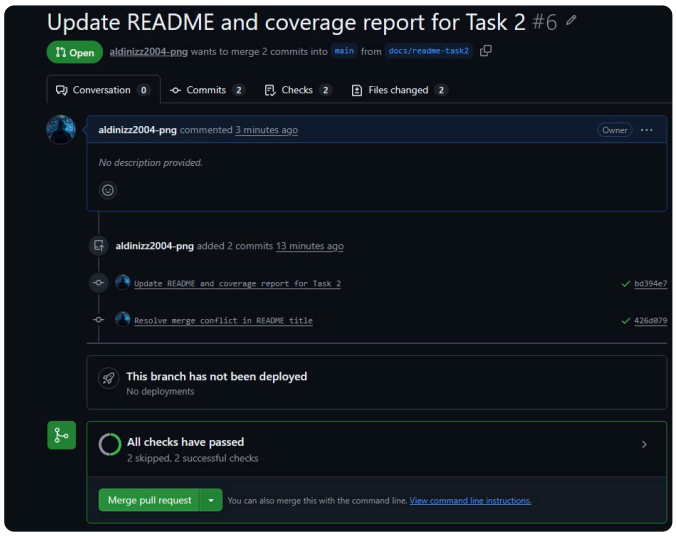
```
# resolve locally on the feature branch
git checkout docs/readme-task2
git fetch origin
git merge origin/main
CONFLICT (content): Merge conflict in README.md
# edit the file, keep the right version, remove
markers
git add README.md
git commit -m "Resolve merge conflict in README
title"
git push
```

Decision

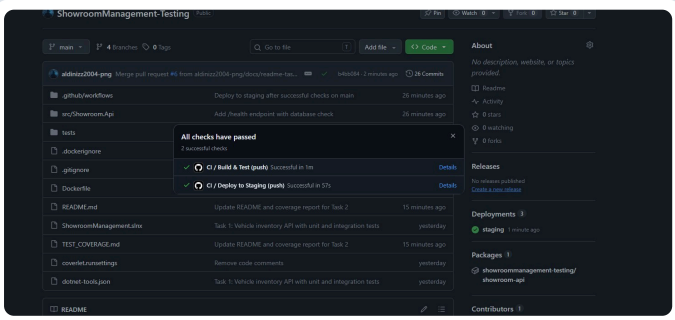
The branch version was kept: it has the Task 2 title and the full technology line (with GitHub Actions and Docker), which the version on main was missing. The conflict markers were removed and only one title line remains.



VS Code: `<<<<<<< HEAD` (branch) vs `>>>>>>> origin/main` .



Resolved: new commit on the PR, all checks passed, merge enabled.



After merging PR #6: both pipeline jobs pass on `main` and the new version is deployed to staging.

Result

The conflict was resolved on the feature branch, not on `main` . The resolution commit went through the same checks as any other change, and only then was the PR merged. `main` stayed green the whole time, and the merge triggered a new staging deployment automatically.

6 CI/CD pipeline (GitHub Actions)

Trigger
every push to any branch, every PR to `main`, manual run

→

Build & Test
checkout · .NET 10 · NuGet cache · restore · build (Release) · unit tests · integration tests · upload `.trx`

→ main only after success

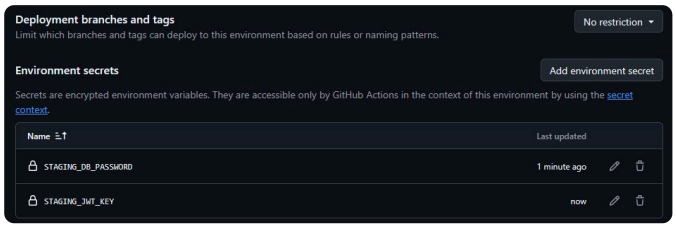
Deploy to Staging
build Docker image · push to `ghcr.io` · start SQL Server 2022 + API · apply migrations

→

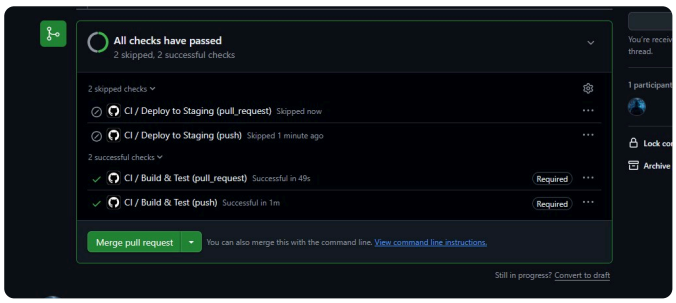
Smoke test
`/health` = Healthy · `/api/vehicles` without token = 401 · summary on the run page

```yaml
deploy-staging:
  needs: build-and-test
  if: github.event_name == 'push' && github.ref ==
'refs/heads/main'
  environment: { name: staging }
  permissions: { contents: read, packages: write }
  env:
    DB_PASSWORD: ${{ secrets.STAGING_DB_PASSWORD }}
    JWT_KEY:     ${{ secrets.STAGING_JWT_KEY }}
```

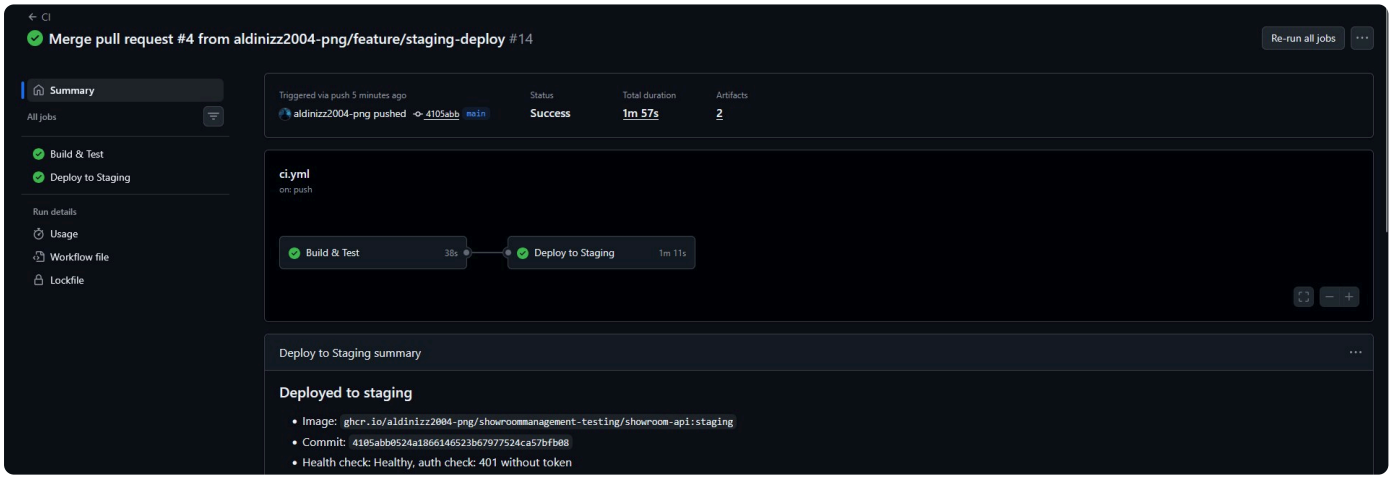
- **Fail fast:** any failing test fails **Build & Test**; the deploy job never starts and the PR cannot be merged.
- **Branches and PRs** run tests only; deploy shows as *skipped*.
- **Secrets** live only in the `staging` environment, never in code, image or logs.
- Image tags: `:staging` and `:sha-<commit>`, so every deployment can be traced to a commit.



Environment "staging" with the two encrypted secrets used by the deploy job.



On a PR: Build & Test is required and passes; Deploy to Staging is skipped.



Successful run on main: Build & Test → Deploy to Staging, with the deployment summary (image, commit, health and auth checks).

7 Deliverables

Required	Where
Updated API	Customers and reservations, role- and resource-based access, `/health`, Dockerfile, migrations `AddCustomers` and `AddReservations`
Extended tests	190 tests (111 unit + 79 integration), all passing; `TEST_COVERAGE.md`
Branches, PRs, review	PRs #1 to #6, review on #1, ruleset on `main`, merge conflict in #6
Successful CI run	Actions tab: Build & Test and Deploy to Staging green on `main`
Documentation	`README.md`: endpoints, roles, run steps, Git workflow, pipeline

Repository: github.com/aldinizz2004-png/ShowroomManagement-Testing · Pull requests: /pulls?q=is:pr · Pipeline runs: /actions